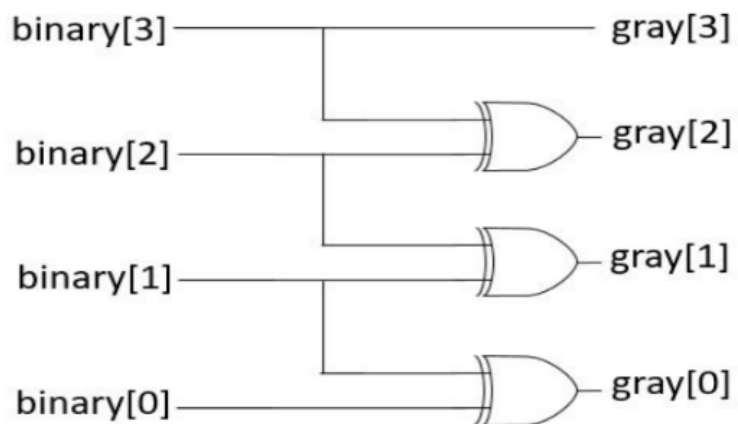


B3	B2	B1	B0	Binary code	G3	G2	G1	G0	Gray code
0	0	0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0	1	1
0	0	1	0	2	0	0	1	1	3
0	0	1	1	3	0	0	1	0	2
0	1	0	0	4	0	1	1	0	6
0	1	0	1	5	0	1	1	1	7
0	1	1	0	6	0	1	0	1	5
0	1	1	1	7	0	1	0	0	4
1	0	0	0	8	1	1	0	0	12
1	0	0	1	9	1	1	0	1	13
1	0	1	0	10	1	1	1	1	15
1	0	1	1	11	1	1	1	0	14
1	1	0	0	12	1	0	1	0	10
1	1	0	1	13	1	0	1	1	11
1	1	1	0	14	1	0	0	1	9
1	1	1	1	15	1	0	0	0	8



```
//behav
module binaryTogray(
    input wire [3:0] binary,
    output reg [3:0] gray
);

always@(*)
begin
    case(binary)
        4'b0000: gray = 4'b0000;
        4'b0001: gray = 4'b0001;
        4'b0010: gray = 4'b0011;
        4'b0011: gray = 4'b0010;
        4'b0100: gray = 4'b0110;
        4'b0101: gray = 4'b0111;
        4'b0110: gray = 4'b0101;
        4'b0111: gray = 4'b0100;
        4'b1000: gray = 4'b1100;
        4'b1001: gray = 4'b1101;
        4'b1010: gray = 4'b1111;
        4'b1011: gray = 4'b1110;
        4'b1100: gray = 4'b1010;
        4'b1101: gray = 4'b1011;
        4'b1110: gray = 4'b1001;
        4'b1111: gray = 4'b1000;
    endcase
end
endmodule
```

```
//dataflow
module Binary2Gray(
    input wire [3:0] binary,
    output wire [3:0] gray
);

assign gray[3] = binary[3];
assign gray[2] = binary[3] ^ binary[2];
assign gray[1] = binary[2] ^ binary[1];
assign gray[0] = binary[1] ^ binary[0];

endmodule
```